

Hyperloop Pathfinding

Algorithm Analysis

Current Algorithms · Limitations · Recommendations · Testing Plan

Austin, TX

Test Map

540 × 864 px

Raster Size

4 (+ 4 recommended)

Algorithms Compared

MHA* 1.01s

Best Speed

Problem Setup

What are we routing, and how is the map structured?

The Raster Grid

- Map is a 2D raster where each cell = 1 pixel of geographic terrain
- Cell value 1 = road/rail corridor | Cell value 0 = off-road terrain
- Movement: 8-directional (N, NE, E, SE, S, SW, W, NW) with diagonal support

Cost Model

On-road traversal	1.0	Standard movement along road network
Off-road traversal	5.0	5× penalty for leaving the road corridor
Terrain roughness	0.12-1.0	Local road density (5×5 window), precomputed

Austin Test Configuration

Start Station:	station1 → pixel (829, 34) → row=34, col=829
End Station:	station2 → pixel (152, 510) → row=510, col=152
Map Bounds:	Lat 30.238-30.285 Lon -97.795 to -97.704 (EPSG:3083)
Map Size:	864 wide × 540 tall = 466,560 total cells
Goals tracked via:	Bitmask: 00=neither, 01=station1 visited, 11=both visited (done)

Current Algorithms — At a Glance

Four algorithms benchmarked on the same Austin test raster

Algorithm	Type	Objectives	Heuristics	Expansions	Time	Path (wp)
A*	Optimal	1 (distance)	1 admissible (Euclidean+MST)	211,023	2.60 s	979
ARA*	Anytime	1 (distance)	1 admissible (inflated $\epsilon \cdot h$)	1,276,755	10.30 s	909
MHA*	Bounded	1 (distance)	3 (anchor+2 inadmissible)	79,775	1.01 s	924
NAMOA*-dr	Pareto	3 (dist, offroad, rough)	3 vector (BFS+Dijkstra)	1,154,829	361.54 s	902 (5 solutions)

Key Insights: MHA* is 4× faster than A* and 350× faster than NAMOA*-dr. NAMOA*-dr is the only algorithm that returns multiple trade-off paths (Pareto front).

A* (A-Star)

Optimal single-source shortest path with admissible heuristic

How It Works

- Maintains a priority queue ordered by $f(n) = g(n) + h(n)$
- $g(n)$ = exact cost from start to n | $h(n)$ = heuristic lower bound to goal
- Expands the node with lowest f first; guarantees optimal path when h is admissible
- Multi-goal handled with bitmask: state = (position, visited_mask)

Austin Results

211,023

Expansions

2.60 s

Time

979 wp

Path

Heuristics

- h_{anchor} : Euclidean distance to nearest unvisited goal
- Plus MST cost of remaining goals (Prim's algorithm)
- Combined: $h = \text{min_dist_to_goal} + \text{mst_cost}(\text{remaining_goals})$
- Always admissible (never overestimates) → guarantees optimal solution

Strengths

- Provably optimal path
- Simple, well-understood
- Fast heuristic (Euclidean+MST)

Guaranteed

Optimality

Weaknesses

- Explores too broadly near obstacles
- Grid-constrained (staircase) paths

Parameters: Diagonal: Yes (8-dir) | On-road cost: 1.0 | Off-road cost: 5.0 | Max expansions: 5,000,000

• No backtracking behavior — must finish before returning answer

ARA* (Anytime Repairing A*)

Returns a good solution fast, then refines it toward optimality over time

How It Works

- Runs weighted A* with inflated heuristic: $f = g + \epsilon \cdot h$ (ϵ starts high)
- High $\epsilon \rightarrow$ finds a path quickly but suboptimal (up to $\epsilon \times$ optimal cost)
- Decreases ϵ each iteration and repairs the solution using INCONS set
- Stops when $\epsilon=1$ (optimal) or time budget exceeded — returns best found so far

Austin Results

1,276,755

Expansions

10.30 s

Time

909 wp

Path

Heuristics

- Same admissible heuristic as A* (Euclidean + MST)
- Inflated by ϵ : $f = g + \epsilon \cdot h$ gives bounded suboptimality guarantee
- $\epsilon=3.0 \rightarrow \text{path} \leq 3 \times \text{optimal}$ | $\epsilon=1.0 \rightarrow \text{path is optimal}$
- Reuses g-values across iterations — no wasted work

Strengths

- Returns first path in $<1s$ ($\epsilon=3$)
- Provable suboptimality bound at each step
- Final result matches A* optimal

10 (3 improving)

Iterations

Weaknesses

- 1.27M total expansions — heaviest search
- Later iterations do redundant work

Parameters: ϵ start: 3.0 | ϵ end: 1.0 | ϵ step: -0.5 | Max iterations: 10 | Max expansions/iter: 2,000,000

MHA* (Multi-Heuristic A*)

Three parallel searches sharing state — fastest algorithm in the comparison

How It Works

- Maintains 3 separate OPEN lists, one per heuristic (anchor + 2 inadmissible)
- Shared g-values and came_from: any heuristic can benefit from another's expansions
- Expansion rule: expand from inadmissible i if $key_i \leq w2 \times anchor_min_key$
- Otherwise expand anchor — this preserves the w1-suboptimality guarantee

Austin Results

79,775
Expansions

1.01 s
Time

924 wp
Path

Heuristics

- H0 Anchor (admissible): Euclidean + MST — guarantees w1-bounded optimality
 - Fastest: 79,775 expansions, 1.01s
- H1 Manhattan+MST (inadmissible): more aggressive, ignores diagonal savings
 - Multiple heuristics explore diverse corridors
- H2 Greedy Road-Biased: $2 \times (\text{Euclidean} + \text{MST})$, discounted $0.8 \times$ when on road cell
 - Bounded suboptimality guarantee ($2 \times$ optimal)
- $w1=2.0$ (suboptimality bound) | $w2=1.2$ (inadmissible expansion threshold)

Strengths

54,949
Anchor expns

Weaknesses

- Single-objective only (distance)
- Greedy Road heuristic fired 0 times — needs tuning

Parameters: w1: 2.0 | w2: 1.2 | Heuristics: 3 | Max expansions: 5,000,000

- w2 tuning sensitive: too high → Manhattan dominates; too low → only Anchor runs

NAMOA*-dr (Non-dominated Multi-Objective A* — Delayed Reopening)

Finds ALL Pareto-optimal paths across 3 objectives simultaneously

How It Works

- Each state holds a set of non-dominated cost vectors (labels), not a single g-value
- A label g_1 dominates g_2 if $g_1[i] \leq g_2[i]$ for all i and $g_1[j] < g_2[j]$ for some j
- Delayed reopening: dominated labels batched and reopened periodically (not immediately)
- ϵ -dominance ($\epsilon=0.15$): g_1 ϵ -dominates g_2 if $g_1[i] \leq (1+\epsilon) \cdot g_2[i]$ — controls label explosion

Austin Results

1,154,829

Expansions

361.54 s

Time

902 wp

Path (selected)

Heuristics

- H_distance: Euclidean + MST (same as A*) — lower bound on remaining distance
 - Only algorithm returning multiple trade-off paths
- H_offroad: precomputed via reverse 0-1 BFS from each goal — exact min off-road cells
 - Complete Pareto front with ϵ -approximation
- H_roughness: precomputed via reverse Dijkstra from each goal — exact min roughness sum
 - Precomputed heuristics give tight bounds
- All three components are admissible → guarantees complete Pareto front

Strengths

5

Pareto solutions

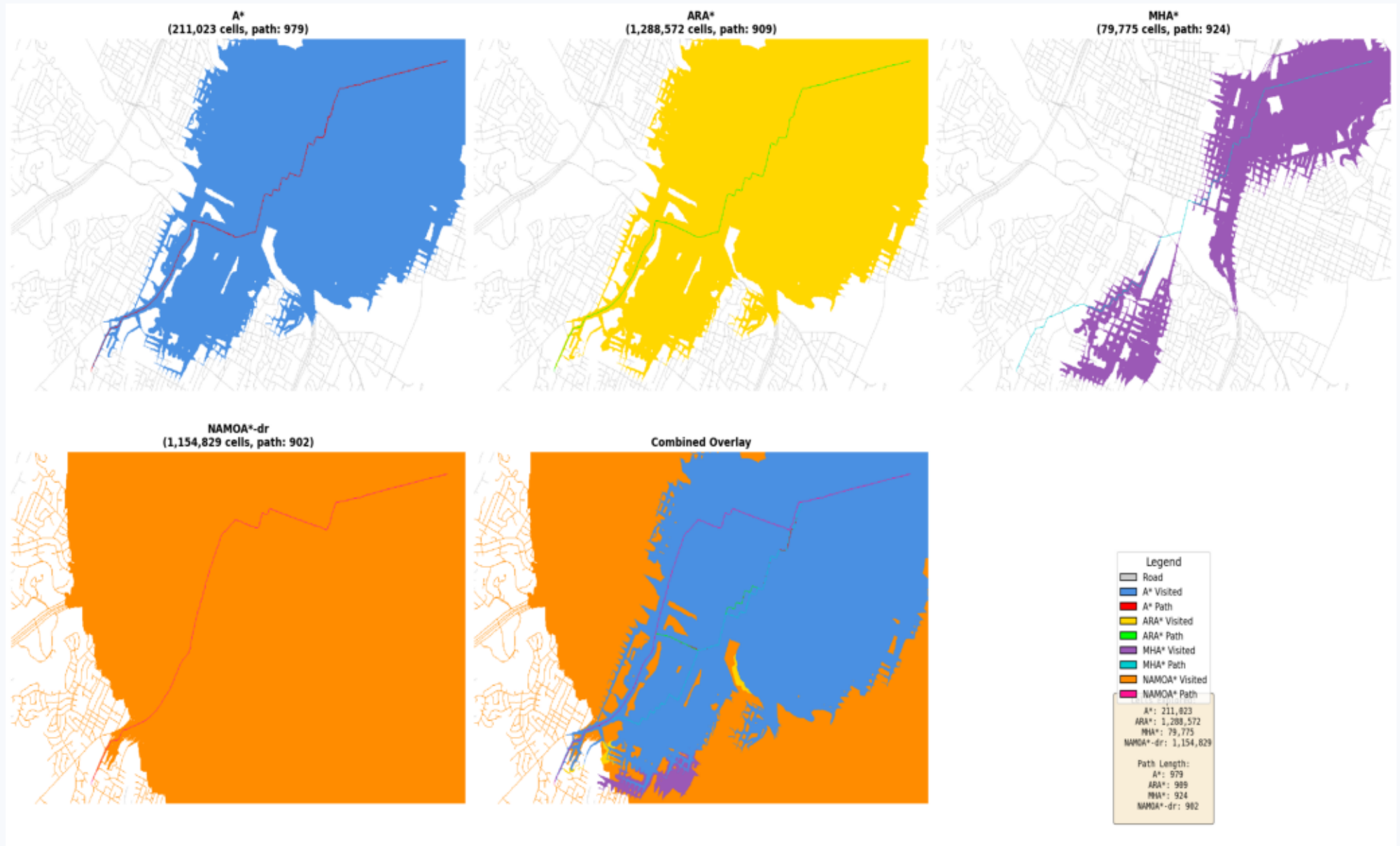
Weaknesses

- 361 seconds (6 min) on small map
- Label explosion even with ϵ -dominance

Parameters: ϵ : 0.15 | Roughness window: 5×5 | Max expansions: 1,500,000 | Objectives: 3 (dist, offroad, rough) | Austin→Dallas scale without major improvement

Visual Comparison — All Four Algorithms

Austin test raster | 2x2 panel layout | each algorithm in its own panel



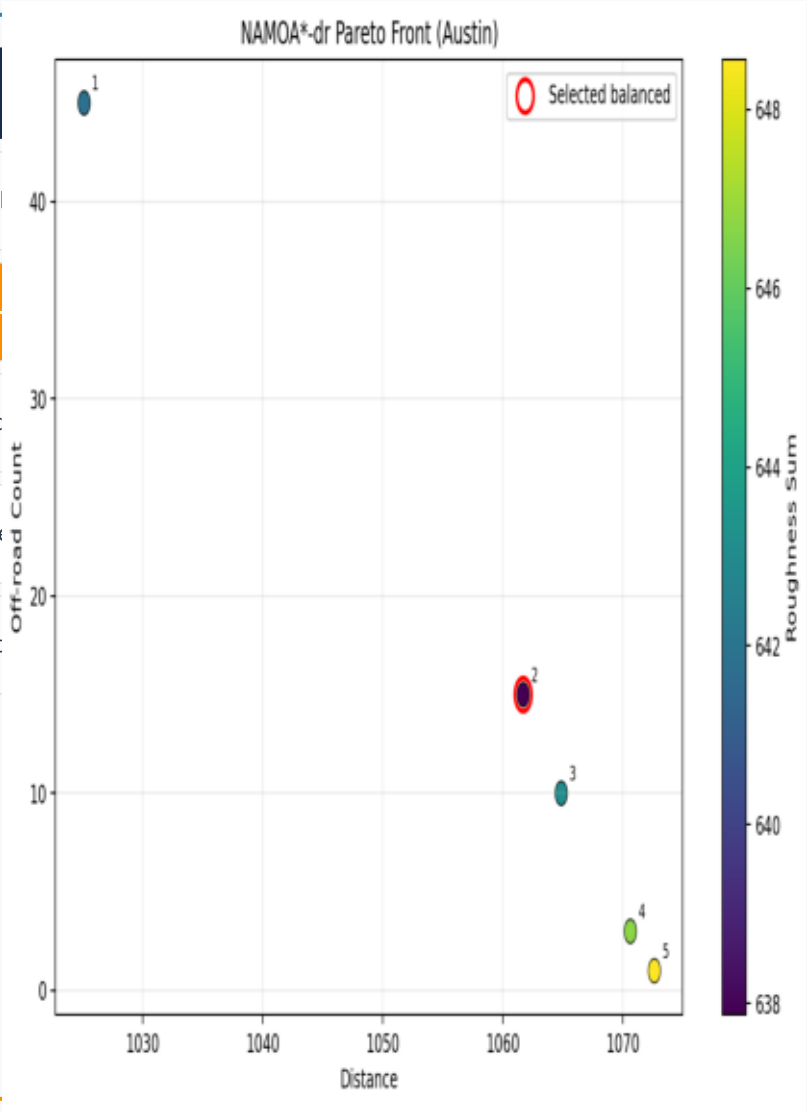
Blue = A* explored | Yellow = ARA* explored | Purple = MHA* explored | Orange = NAMOA*-dr explored · Bright path lines show final selected route for each algorithm

NAMOA*-dr — Pareto Front

Five trade-off paths found; user selects based on priorities

The 5 Pareto-Optimal Solutions

#	Distance (units)	Off-Road Cells	Roughness Sum	Waypoints	Status
1	1025.1	45	641.84	865	Shortest path
2	1061.7	15	637.88	902	SELECTED - balanced
3	1064.9	10	643.08	906	10 off-road crossings
4	1070.6	3	646.72	913	Almost entirely on-road
5	1072.6	1	648.56	915	Most on-road



How to Read the Pareto Front

No single solution is 'best' — each is optimal for a different priority. A path with fewer off-road crossings is always longer. The 'balanced' selection (Sol. 2) minimizes the normalized distance to the ideal point (0, 0, 0).

Limitations of Current Algorithms

What these four algorithms cannot do well — and why it matters for hyperloop

Grid-Constrained Movement

- All 4 algorithms snap to 8 grid directions: N, NE, E, SE, S, SW, W, NW
- Real hyperloop tubes travel in any continuous direction — not 45° increments
- Result: staircase-like paths with unnecessary turns, 5-10% longer than necessary

Scale — Austin → Dallas

- Current test raster: 540×864 = 466,560 cells (suburb-scale)
- Austin → Dallas: ~195 miles. At same resolution → ~10M+ cells
- A* would need tens of millions of expansions; NAMOA*-dr would be infeasible

NAMOA*-dr Runtime

- 361 seconds (6 min) on a small 540×864 map with ε-dominance at 0.15
- Multi-objective label explosion grows exponentially with map size
- Not practical for interactive planning or larger city-pair corridors

MHA* Greedy Road Heuristic Underperforms

- The 3rd heuristic (Greedy Road-Biased) fired 0 times out of 79,775 expansions
- Manhattan+MST always won the expansion race — GreedyRoad never got a turn
- Opportunity: better inadmissible heuristic design could improve further

A* path: ↗ ↗ → → → ↘ ↘ ↘ → →
Theta*: _____

Current: 540×864 → 466K cells
A→D map: ~3,000×4,000 → 12M cells

Austin small map: 361 s
Estimated A→D map: hours

Anchor: 54,949 expansions
Manhattan: 24,826 expansions
GreedyRoad: 0 expansions

Recommendation: Theta*

Any-angle pathfinding — removes the grid-direction constraint entirely

Problem Solved: All current algorithms snap to 8 directions, producing staircase paths 5–10% longer than necessary.

How It Works

- During neighbor relaxation, try shortcutting through the grandparent node
- If `line_of_sight(grandparent, neighbor)` is clear → connect directly
- Result: paths follow natural straight lines through open terrain

$$f(n) = g(\text{parent_or_grandparent}) + \text{dist}(\text{parent_or_grandparent}, n) + h(n)$$

Expected Benefits

- Paths 5–10% shorter in Euclidean distance
- Dramatically fewer direction changes (critical for hyperloop construction cost)
- Same time complexity as A* — no meaningful slowdown
- Lazy Theta* variant even faster on dense grids

Worked Example

```
Standard A*:
  relax(parent → current → neighbor)

Theta* change:
  if line_of_sight(grandparent, neighbor):
    relax(grandparent → neighbor) ← skip intermediate cells
  else:
    relax(parent → neighbor) ← fallback to A*
```

Implementation Effort

LOW — ~30 line change to A* neighbor loop + line_of_sight helper

vs. Current Algorithms

vs A*: same optimality class, shorter and smoother paths
vs MHA*: can be combined → Multi-Heuristic Theta*
vs NAMOA*: can extend to multi-objective Theta* with cost vectors

Recommendation: Bidirectional A*

Meet-in-the-middle search — halves the effective search depth

Problem Solved: Single-direction search must traverse the entire Austin–Dallas corridor. For long-distance routes, this wastes the majority of expansions on empty space.

How It Works

- Run two simultaneous A* frontiers: Forward (from Austin) and Backward (from Dallas)
- Each iteration expands the frontier with the smaller f-value
- Stop when the two frontiers overlap; the meeting point yields the optimal path

Stop when: $g_{\text{fwd}}(u) + g_{\text{bwd}}(u) \leq \text{best_path_cost}$ for any node u

Expected Benefits

- 50–70% fewer expansions on long corridors
- Pairs with any heuristic — use same Euclidean+MST
- Can combine with Theta* → Bidirectional Theta*
- Critical for scaling to full Austin → Dallas distance

Worked Example

Forward search: Austin → meeting point
Backward search: Dallas → meeting point

Search space comparison:
One-way A*: $O(b^d)$ b =branching, d =depth
Bidirectional: $O(b^{(d/2)})$ ← square root reduction

For $d=1000$ expansions needed:
A*: 1,000 expansions
BiA*: ~32 expansions from each end = ~64 total

Implementation Effort

MEDIUM — two open lists, backward heuristic, termination condition

vs. Current Algorithms

vs A* (211K expns): expect ~70–120K expansions
vs MHA* (79K expns): comparable, different trade-off
Combine both: Bidirectional MHA* could be even faster

Recommendation: EMOA*

Enhanced Multi-Objective A* (2022) — same Pareto front, far fewer expansions

Problem Solved: NAMOA*-dr takes 361 seconds on a small map due to label explosion. This makes multi-objective routing infeasible at scale.

How It Works

- Labels processed in sorted order of first objective (distance) using a priority queue
- Tighter dominance pruning: uses G-closed sets indexed by first objective
- Detects and discards dominated labels earlier — before they enter the open list
- Result: exponentially fewer labels survive to expansion

key(label) = (g[0], g[1], g[2]) sorted by g[0] first

Expected Benefits

- Same complete Pareto front guarantee as NAMOA*-dr
- 10–40% of NAMOA*-dr expansions (literature benchmarks on road networks)
- Same ϵ -dominance extension possible for approximate Pareto
- Published 2022 — state-of-the-art multi-objective pathfinding

Implementation Effort

MEDIUM — replaces NAMOA*-dr; same data interface and output format

Worked Example

NAMOA*-dr label check (per expansion):
Compare new label against ALL labels in closed set
→ $O(|\text{closed}|)$ per expansion

EMOA* label check:
Closed set indexed by objective 1 (sorted)
Binary search prunes majority of dominated labels
→ $O(\log|\text{closed}|)$ per expansion

Benchmark results (literature):
NAMOA*-dr: 100% baseline
EMOA*: 10–40% of NAMOA* expansions on road networks

vs. Current Algorithms

vs NAMOA*-dr (361s): expect 40–150s on same Austin map
Same 5 Pareto solutions, same output format
Direct drop-in replacement in `compare_all_algorithms.py`

Recommendation: ALT (A* + Landmarks + Triangle Inequality)

Precomputed landmark distances give tight heuristics — critical for large maps

Problem Solved: Euclidean distance heuristic is weak on maps with obstacles (rivers, protected areas). For a full Austin→Dallas raster (12M+ cells), A* expansions would be unacceptable.

How It Works

Expected Benefits

- Choose ~16 landmark nodes spread across the map (corners, key waypoints)
 - Precompute exact shortest distances from every cell to every landmark (Dijkstra × 16)
 - At query time: $h(u, goal) \geq |d(landmark, goal) - d(landmark, u)|$ for each landmark
 - Take the maximum across all landmarks → tightest admissible lower bound
- 2–5× fewer A* expansions vs Euclidean heuristic on obstacle-heavy maps
 - One-time preprocessing — reusable for all Austin↔Dallas queries
 - Works with any single-objective algorithm (A*, ARA*, MHA*, Theta*)
 - Standard technique in OpenStreetMap (OSRM) and geographic routing
- $h_{ALT}(u, goal) = \max \text{ over all landmarks } L \text{ of } |d(L, goal) - d(L, u)|$**

Implementation Effort

MEDIUM — precomputation script + modified heuristic function

Worked Example

Triangle inequality:
 $d(u, goal) \geq d(L, goal) - d(L, u)$ for any landmark L
 $d(u, goal) \geq d(u, L) - d(goal, L)$

Example on Austin map:
Euclidean h: 200 cells (weak lower bound)
ALT h: 195 cells (tighter — avoids known detours)

Preprocessing:
Run 16 Dijkstras once → save distance arrays
Reuse for every future query on same map
Preprocessing: ~minutes | Query speedup: 2–5×

vs. Current Algorithms

vs A* (211K expns): expect 50–100K with good landmarks
vs MHA*: can replace all 3 heuristics with ALT variants
Most impactful when scaling to Austin→Dallas full raster

Testing Plan

How to implement, benchmark, and validate each recommended algorithm

Implementation Pattern (same for every new algorithm)

1	Create	hyperloop_<name>_from_npz.py (same CLI interface as existing algorithms)
2	Register	Add run_<name>() to compare_all_algorithms.py and run_all_tests.py
3	Animate	Extend astar_animator.py — new panel in the 2×2 (or 3×2) grid GIF
4	Run Austin	python compare_all_algorithms.py --input austin_test_raster.npz (fast feedback)
5	Scale	Run on Seattle and Portland once Austin results look correct

Metrics to Record per Algorithm

Expansions:	Total nodes popped from priority queue
Wall-clock time:	End-to-end runtime including heuristic precomputation
Path length:	Number of waypoints in final path
Path smoothness:	Count of direction changes (fewer = smoother = better for hyperloop)
Path cost:	Sum of edge weights along the final path
Off-road cells:	Number of raster cells in path with road_bitmap=0 (NAMOA / EMOA only)

Visual Validation Checklist

- ☐ Path connects station1 (top-right) to station2 (bottom-left) — no disconnected blobs
- ☐ Exploration heatmap covers a corridor between the two stations — not a random blob
- ☐ Path color distinct in the 2×2 grid animation (no color overlap between panels)
- ☐ NAMOA* / EMOA* Pareto front plot shows increasing trade-off (not all solutions identical)
- ☐ Theta* paths visibly smoother than A* paths — fewer diagonal staircase segments

Summary & Recommendation

Which algorithm to use and when — for Austin–Dallas hyperloop routing

Current Algorithm Scorecard

A*	✓ Optimal	✗ Grid-constrained paths	Baseline — replace with Theta*
ARA*	✓ Anytime	✗ 1.27M expansions, 10s	Good for time-limited search
MHA*	✓ Fastest (1s)	✗ Single-objective only	Best speed; add Theta* variant
NAMOA*-dr	✓ Pareto front	✗ 361s — very slow	Replace with EMOA* for scale

Recommended Additions (Priority Order)

1	Theta*	Any-angle	LOW effort	Drop-in A* replacement. Smoother paths, same runtime. Most impactful for infrastructure planning.
2	Bidirectional A*	Speed	MEDIUM effort	Search from both ends simultaneously. ~50–70% fewer expansions for long Austin→Dallas corridors.
3	EMOA*	Multi-obj	MEDIUM effort	Replaces NAMOA*-dr. Same Pareto front guarantee with dramatically fewer expansions.
4	ALT	Scale	MEDIUM effort	Landmark-based heuristic precomputation. Enables scaling to full city-pair rasters.

Bottom line: Implement Theta* first — it improves every other algorithm and requires minimal code change. Then EMOA* to fix the NAMOA*-dr runtime problem.